Experimental Report on the School of Computer and Artificial Intelligence at Zhengzhou University

Grade：2024 Major：SOFTWARE ENGINEERING Class：GROUP-1

Name：MD TASNIMUL HASAN Student Number：202480090104

Instructor：ALEX Score： 2026 year 10 month 26 date

Experimental Topic：Design and Implementation of Implicit Drawer, Zoom Control, and Popup Window in Android

1. Experimental Purpose

- Master the basic usage of SlidingDrawer (sliding drawer) for creating a collapsible panel with a handle and content area.
- Understand and use ZoomControls to provide simple zoom-in and zoom-out buttons with callback events.
- Master the creation and display of PopupWindow as a floating container anchored to a view.

2. Experimental Content

1.SlidingDrawer

- Design a vertical SlidingDrawer containing a handle (a button) and a content area (e.g., a set of quick-action buttons or an image).
- Demonstrate opening and closing the drawer programmatically via open() and close() methods.
- Note: SlidingDrawer is a legacy widget and may require careful handling on modern Android versions.

2.ZoomControls

- Place a ZoomControls widget on the screen to provide zoom-in and zoom-out buttons.
- Register setOnZoomInClickListener and setOnZoomOutClickListener to handle zoom actions

(e.g., changing the text size of a TextView or scaling an image).

- Note: ZoomControls is deprecated; the experiment serves to understand its historical usage and simple callback pattern.

3.PopupWindow

- Create a PopupWindow containing a simple layout (e.g., a TextView or a small menu).
- Display it anchored to a button using showAsDropDown(View anchor).
- Configure basic properties such as width, height, focusability, and outside-touch dismissal.

3. Experimental Environment

Item	Specification
OS	Windows 10 / 11 (or macOS)
IDE	Android Studio (Giraffe / Hedgehog or later)
Language	Kotlin
SDK	Android SDK API 24 – 34 (minSdk 24, targetSdk 34)
Emulator	Android Emulator (Pixel, API 30+) or physical device
Build System	Gradle

Note on Deprecation: SlidingDrawer and ZoomControls are both deprecated in modern Android development. This experiment is a verification exercise to understand their basic APIs. In real projects, DrawerLayout and custom zoom controls are recommended alternatives

4. Procedure

Step 1: Create the Project

1. Open Android Studio and create a New Project using the Empty Views Activity template.
2. Set the project name to DrawerZoomPopupDemo, choose Java as the language, and set the minimum SDK to API 24.
3. Wait for the Gradle sync to complete.

Step 2: Design the Main Layout

1. Open activity_main.xml.
2. Add a vertical LinearLayout as the root container with padding.

3. Inside the root layout, place:
    - A TextView (id: tvZoomTarget) with sample text (e.g., "Zoom me!") to demonstrate zoom effects.
    - A ZoomControls widget (id: zoomControls) below the text.
    - A Button (id: btnShowPopup) labeled "Show Popup Window".
    - A Button (id: btnOpenDrawer) labeled "Open Drawer".

4.Add a SlidingDrawer as the last child of the root layout, with:

- android:handle="@+id/handle" pointing to a Button labeled "Handle".
- android:content="@+id/content" pointing to a LinearLayout containing a few sample Buttons or a TextView as the drawer content.
- android:orientation="vertical".

Step 3: Implement ZoomControls Logic

1. In MainActivity.java, declare member variables for the TextView and ZoomControls.
2. In onCreate(), bind these views.
3. Set setOnZoomInClickListener on the ZoomControls to increase the TextView's text size by a fixed amount (e.g., +2sp).
4. Set setOnZoomOutClickListener to decrease the text size by the same amount, with a lower bound (e.g., minimum 10sp).
5. Optionally, call setIsZoomInEnabled or setIsZoomOutEnabled to disable buttons when limits are reached.

Step 4: Implement PopupWindow Logic

1. Create a new layout file popup_content.xml containing a simple LinearLayout with a TextView (e.g., "This is a PopupWindow").

2. In MainActivity.java, create a helper method showPopupWindow(View anchor).
3. Inside the method:

- Inflate popup_content.xml to obtain the content view.
- Instantiate a PopupWindow with the content view, setting width and height to WRAP_CONTENT.
- Call setFocusable(true) and setOutsideTouchable(true) so the popup can be dismissed by tapping outside.
- Call showAsDropDown(anchor) to display the popup anchored to the clicked button.

4.Attach a click listener to btnShowPopup that calls showPopupWindow(v).

Step 5: Implement SlidingDrawer Logic

1. In MainActivity.java, bind the SlidingDrawer using findViewById.
2. Attach a click listener to btnOpenDrawer that calls slidingDrawer.open() to programmatically open the drawer.
3. Add a button inside the drawer content (e.g., "Close Drawer") that calls slidingDrawer.close().
4. Optionally, register an OnDrawerOpenListener and OnDrawerCloseListener to show a Toast when the drawer state changes.

Step 6: Run and Test the Application

1. Click **Run** to install the app on the emulator or device.
2. Verify:

- Tapping the **ZoomControls** plus/minus buttons changes the TextView text size.
- Tapping **Show Popup Window** displays the popup anchored below the button; tapping outside dismisses it.
- Tapping **Open Drawer** or dragging the handle opens the SlidingDrawer; tapping the handle or a close button inside closes it.

3.If SlidingDrawer causes issues on the target API level, check Logcat for warnings and note the deprecation behavior in the analysis.

5. Results and Analysis

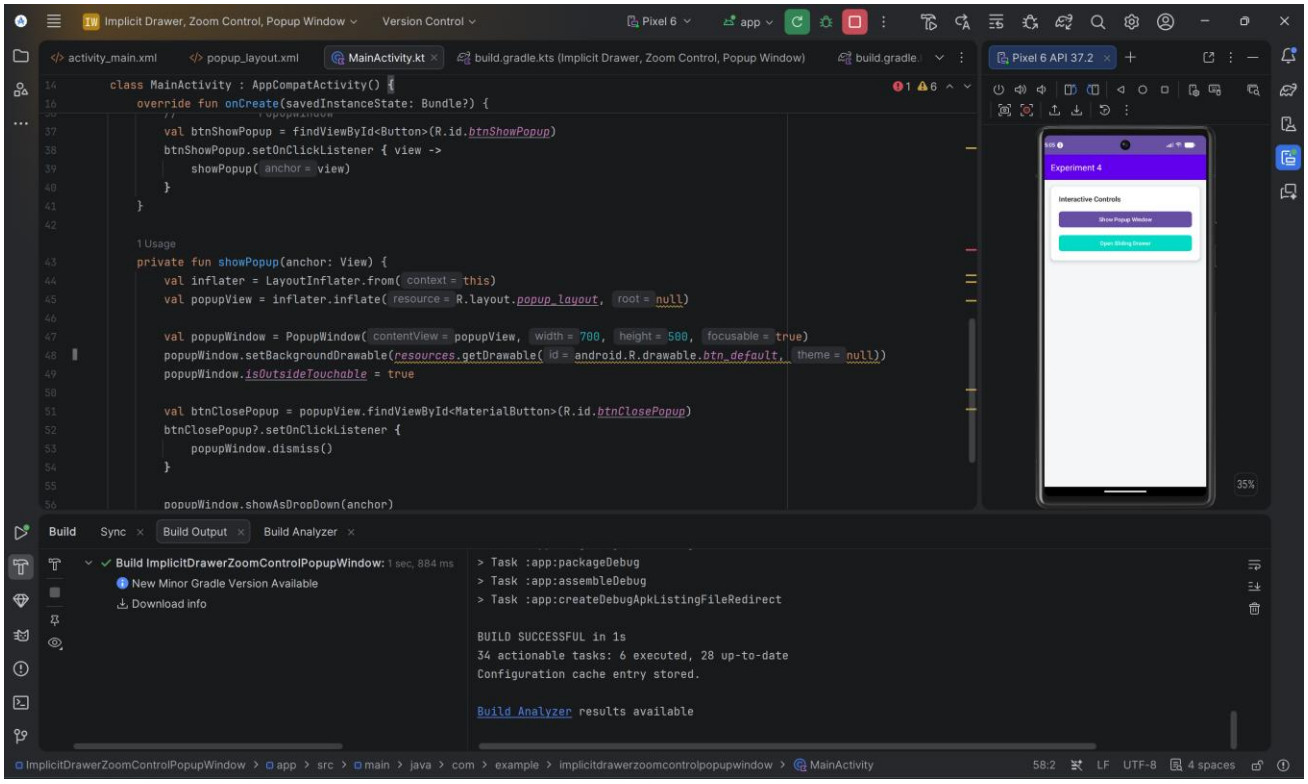


Figure: Windows Interface

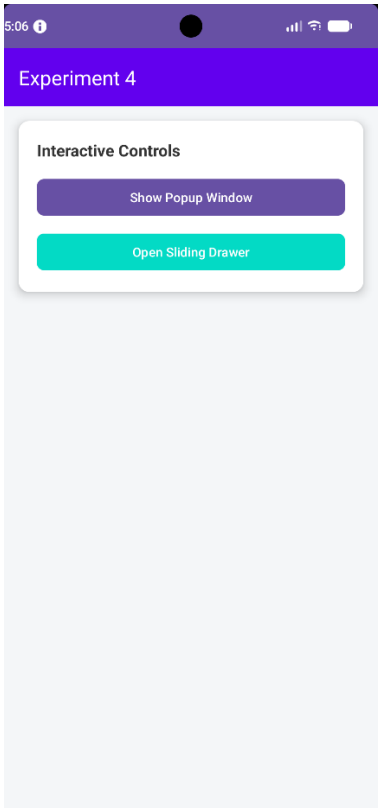


Figure : Main Interface

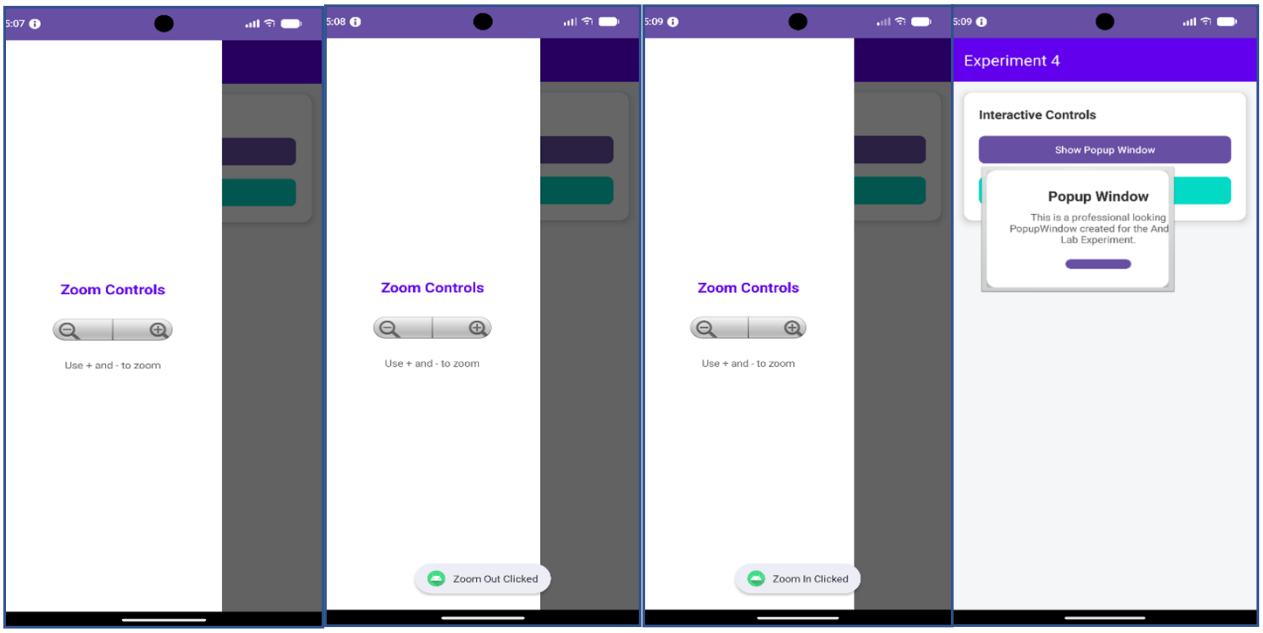


Figure: Drawer,Zoom Control and Popup Window

Results

- ZoomControls successfully triggered text size changes via its zoom-in and zoom-out callbacks; the zoom speed and enabled states could be controlled programmatically.
- PopupWindow displayed correctly anchored to the button using showAsDropDown, and was dismissed when tapping outside (after setting setOutsideTouchable(true)).
- SlidingDrawer opened and closed both by dragging the handle and by programmatic calls to open() / close(); drawer state change listeners fired as expected.

Analysis

- **Deprecation**: Both SlidingDrawer and ZoomControls are officially deprecated. SlidingDrawer has been replaced by NavigationView + DrawerLayout for side menus, and ZoomControls is superseded by custom zoom UI or gesture detection. This experiment verifies their legacy behavior but highlights that new projects should avoid them.
- **PopupWindow vs Dialog**: PopupWindow is lightweight and anchored to a view, making it suitable for context menus or quick info panels; it does not dim the background by default. AlertDialog is modal and centered. Choosing between them depends on the interaction pattern required.

- **SlidingDrawer constraints**: The handle and content attributes are mandatory and must refer to distinct child views; missing either throws an exception at inflation time. This strict requirement makes the widget less flexible than modern alternatives.
- **ZoomControls simplicity**: The callback-based API (setOnZoomInClickListener) is straightforward, but it offers no built-in gesture support (pinch-to-zoom), which is why custom implementations are preferred today.

6. Summary

In this experiment, I practiced three legacy Android UI components:

1. SlidingDrawer — Created a collapsible drawer with a handle and content area, and controlled it both interactively and programmatically. Understood its mandatory handle/content attributes and its deprecation in favor of DrawerLayout.
2. ZoomControls — Used the built-in zoom buttons with click callbacks to modify text size, and explored enabling/disabling zoom buttons at limits.
3. PopupWindow — Built a floating popup anchored to a button, configured focusability and outside-touch behavior, and displayed it with showAsDropDown.

Key takeaways: SlidingDrawer and ZoomControls are deprecated and should not be used in production; however, studying them clarifies how Android evolved toward more flexible components like DrawerLayout and custom gesture handling. PopupWindow remains a useful lightweight alternative to dialogs for anchored floating content.